

Node.js Module

Understanding the Node.js Module System

Self-contained code unit:

- Each file in Node.js is treated as a separate module.
- Variables, functions, or objects defined in one file are not accessible in another file by default unless you explicitly export them.

Encapsulation:

- Node.js uses the CommonJS module system (`module.exports` and `require`) to ensure the code in one file does not pollute or interfere with the global scope.
- This makes your code modular, maintainable, and easier to debug.

What Exactly is a Module in Node.js?

- A module in Node.js represents a file containing code that is self-contained, reusable, and encapsulated.
- Node.js uses the CommonJS module system.
- This module system came before ES Modules was introduced in JavaScript; that's why its syntax is different.
- Modules in Node.js are created by defining separate files for different functionalities.
- You must export anything you want to make accessible to other modules.

THAPA TECHNICAL - NODEJS YOUTUBE SERIES

Creating and Organizing Modules

```
// math.js
const add = (a, b) => a + b;
module.exports = add;

// app.js
const addFunction = require('./math');
console.log(addFunction(2, 3)); // Outputs: 5
```

- To make variables or functions available outside the module, they must be explicitly exported using **module.exports**.
- In another module, the exported content can be imported using **require()**.

Named Export

Key Takeaways

- Avoid mixing `module.exports` and `exports.property` incorrectly. If you reassign `module.exports`, it will override any previous `exports.property` assignments.
- Use consistent syntax for clarity:
 - Assign everything at once with `module.exports = { ... }.`
 - Or assign properties individually with `module.exports.property.`